CHAPTER 1

Introduction to Software Engineering

A comprehensive overview of software engineering principles, concepts, and practices

Overview

Contents

01 **Software Engineering Discipline**
Origin, formation, and establishment

02 **Engineering Principles**
Simple vs complex construction

03 **Case Study**
Library management system design

04 **What's Software?**
Definition, components, documentation

05 **What's Good Software?**
Quality models and perspectives

06 **Characteristics & Categories**
Software properties and types

07 **Evolution & Crisis**
Historical development and challenges

08 **Software Engineering**
Definition, process, tools, myths

01

Software Engineering Discipline Introduction

The origin and development of software engineering as a formal discipline

Historical Origin

Concept Proposal

NATO Conference 1968

The concept of **"software engineering"** was first formally proposed at the NATO Software Engineering Conference in 1968, held in Garmisch, Germany. This landmark conference brought together leading computer scientists from NATO countries to address what was termed the "software crisis."

Original Definition

The representative definition at that time: "Reasonable engineering principles and methods formulated and used to economically obtain reliable software working on real machines."

IEEE Publication 1972

In 1972, the IEEE Computer Society published the first **"Software Engineering Journal,"** marking the formal academic recognition of the discipline.

Key Insight: The term was meant to provoke thought regarding software development practices and establish engineering discipline principles.

DISCIPLINE EVOLUTION

Discipline Formation



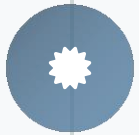
Late 1970s Academic Recognition

The United States included **software engineering courses** in graduate teaching programs, marking the first formal academic recognition of the field. Universities began to recognize the need for structured education in software development methodologies.



Late 1980s - Early 1990s CMU/SEI Support

Software engineering education received significant support from **Carnegie Mellon Software Engineering Institute (CMU/SEI)**, established in 1984. SEI played a crucial role in advancing software engineering practices and establishing standards.



1991 ACM/IEEE-CS Recognition

ACM (Association for Computing Machinery) and **IEEE-CS** (IEEE Computer Society) listed **"Software Engineering"** as one of the nine knowledge areas of computing science.



1993 Joint Committee

IEEE-CS and ACM established the **IEEE-CS/ACM Joint Steering Committee** to build software engineering into a recognized profession, setting curriculum standards and accreditation criteria.

FORMALIZATION

Discipline Establishment

June 2004

IEEE-CS and ACM Joint Release

SWEBOK

Software Engineering Body of Knowledge

A comprehensive framework defining the knowledge areas essential to software engineering practice.

Ten Knowledge Areas

1. Software Requirements
2. Software Design
3. Software Construction
4. Software Testing
5. Software Maintenance
6. Software Configuration Mgmt
7. Software Engineering Mgmt
8. Software Engineering Process
9. Software Quality
10. Tools and Methods

Significance: SWEBOK established software engineering as a legitimate engineering discipline.

It provides a standard framework for curriculum development, professional certification, and industry practice.

COMPUTING SCIENCES

Software Engineering in Computing Sciences

One of Nine Knowledge Areas

Software engineering is a fundamental discipline within computing sciences, integrating principles from mathematics, engineering, and computer science.

1. **Algorithms & Data Structures**
   Fundamental computational methods and data organization
2. **Programming Languages**
   Tools and notation for expressing computation
3. **Computer Architecture**
   Hardware organization and system structure
4. **Numerical & Symbolic Computation**
   Mathematical and symbolic processing methods
5. **Operating Systems**
   System software managing hardware resources

6

Software Engineering

Systematic, disciplined approach to software development, operation, and maintenance throughout the lifecycle.

7. **Database & Information Retrieval**
   Data storage, organization, and access methods
8. **AI & Robotics**
   Intelligent systems and autonomous computation
9. **Human-Computer Interaction**
   Interfaces and interaction between users and systems

Integration: Software engineering integrates principles from all areas, applying systematic engineering approaches to develop reliable, maintainable software products.

02

Software Engineering as Engineering

Understanding software engineering through the lens of traditional engineering principles

ENGINEERING PRINCIPLES

What is Engineering?

Definition of Engineering

Engineering is the application of science and mathematics, through which the properties of natural materials and energy can be structured into machines, products, systems, and processes to create **efficient, reliable, and useful things** for humanity in the shortest time and with minimal human resources.

Engineering disciplines apply theoretical knowledge to practical problems, creating tangible solutions.

Traditional Engineering

- Architectural Engineering
- Mechanical Engineering
- Electrical Engineering

Software Engineering

Applies **engineering, scientific, and mathematical principles** to develop and maintain computer software.

Key Engineering Activities

Research
Systematic investigation

Development
Creating new solutions

Design
Planning and specification

Production
Manufacturing and building

Operations
Running and maintaining

Software Engineering Advantage: Develop high-quality software with lower cost and maintain it effectively through systematic approaches.

ARCHITECTURE ANALOGY

Simple Architecture

Building Simple Structures

Building **simple architecture** only requires basic architectural knowledge. Understanding fundamental elements allows for direct construction with minimal planning.

Basic Elements

- Pillars
- Beams
- Roof
- Walls

Approach: With these basic elements in mind, one can build quickly through mental conception without extensive documentation or formal design processes.



ARCHITECTURE ANALOGY

Complex Architecture



Collaborative Engineering

Complex architecture is the result of collaboration among many people. It is impossible to construct while conceiving. Before actual construction, all design work must be completed and reviewed.

Required Design Work

- **Exterior Design**
- **Graphic Design**
- **Structural Design**
- **Construction Design**
- **Electrical Design**
- **Water & Drainage Design**

Documentation Requirement

Detailed documentation must be left for maintenance during use. This ensures long-term viability and proper upkeep.

ARCHITECTURE DOCUMENTATION

Complex Architecture: Documentation

Essential Documentation for Complex Projects

Complex architectural projects require comprehensive documentation to ensure quality, coordination, and maintainability.

Project Plan

Overall project scope, timeline, and resource allocation

Construction Quality Plan

Quality standards and assurance procedures

Standards & Specifications

Technical standards and material specifications

Construction Manual

Step-by-step construction procedures and safety guidelines

Design Documents

Detailed architectural and engineering drawings

And More...

Maintenance manuals, safety protocols, compliance certificates

Purpose: These documents enable coordination among teams, ensure quality, facilitate maintenance, and provide legal protection.

Key Principle: Design first, build later. Documentation ensures clarity and feasibility before construction.

PROGRAMMING ANALOGY

Simple Program

</>Simple Programs

Simple programs don't require formal design. You can just start coding and complete them quickly.

Design Process: Strictly speaking, the design process is essential—it's just completed rapidly in the mind.

For experienced developers, simple programs require minimal explicit design.

Characteristics

- Single functionality
- Minimal code (dozens of lines)
- No complex data structures
- Single developer can complete

>_Example: Hello World

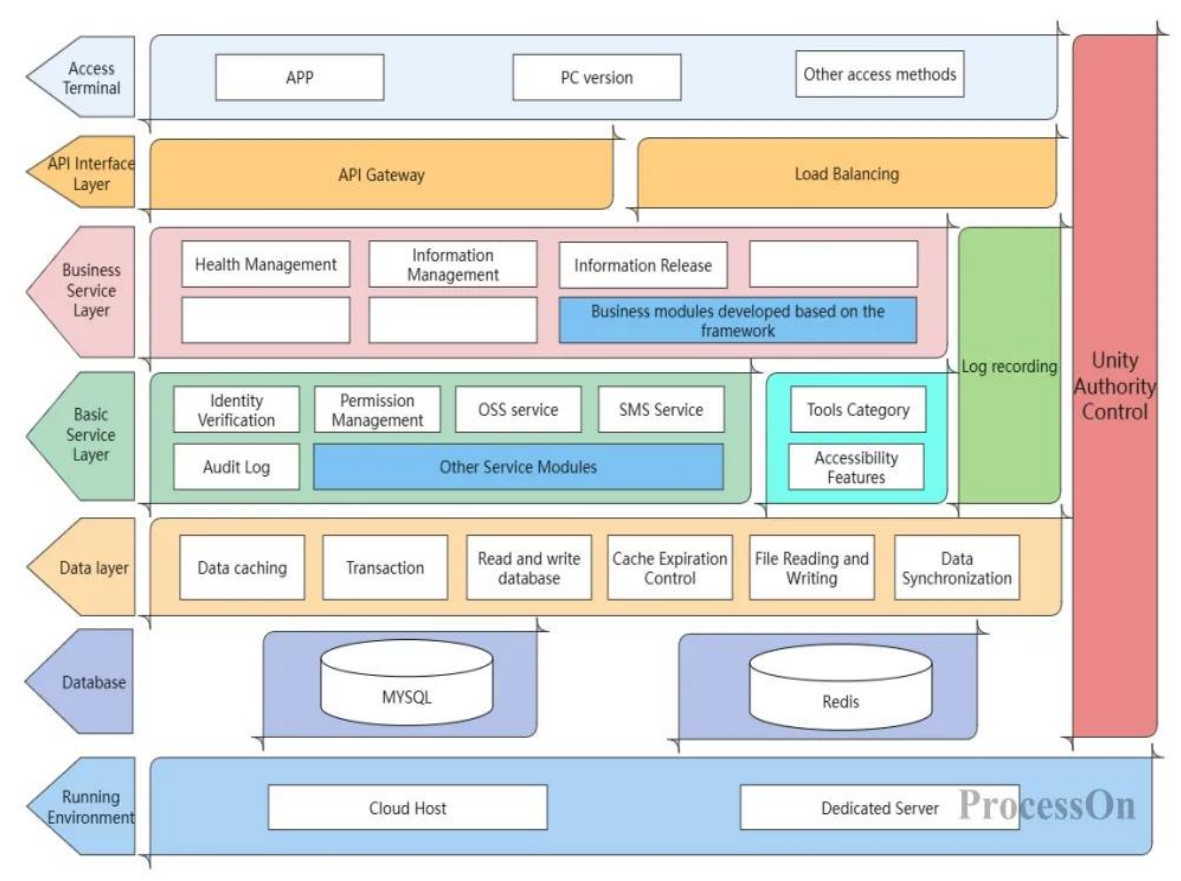
```
// Simple C program
#include <stdio.h>

int main()
{
    printf("Hello, World!");
    return 0;
}
```

- Single function
- One output statement
- No design needed

PROGRAMMING ANALOGY

Complex Program



Complex Software Systems

For **complex functional requirements**, applications require sophisticated design and planning.

Complexity Characteristics

- **Complex Structure:** Multiple layers, components
- **Numerous Modules:** Hundreds of functional units
- **Complex Data:** Sophisticated data structures
- **Massive Code:** Thousands to millions of lines

Team Collaboration Required

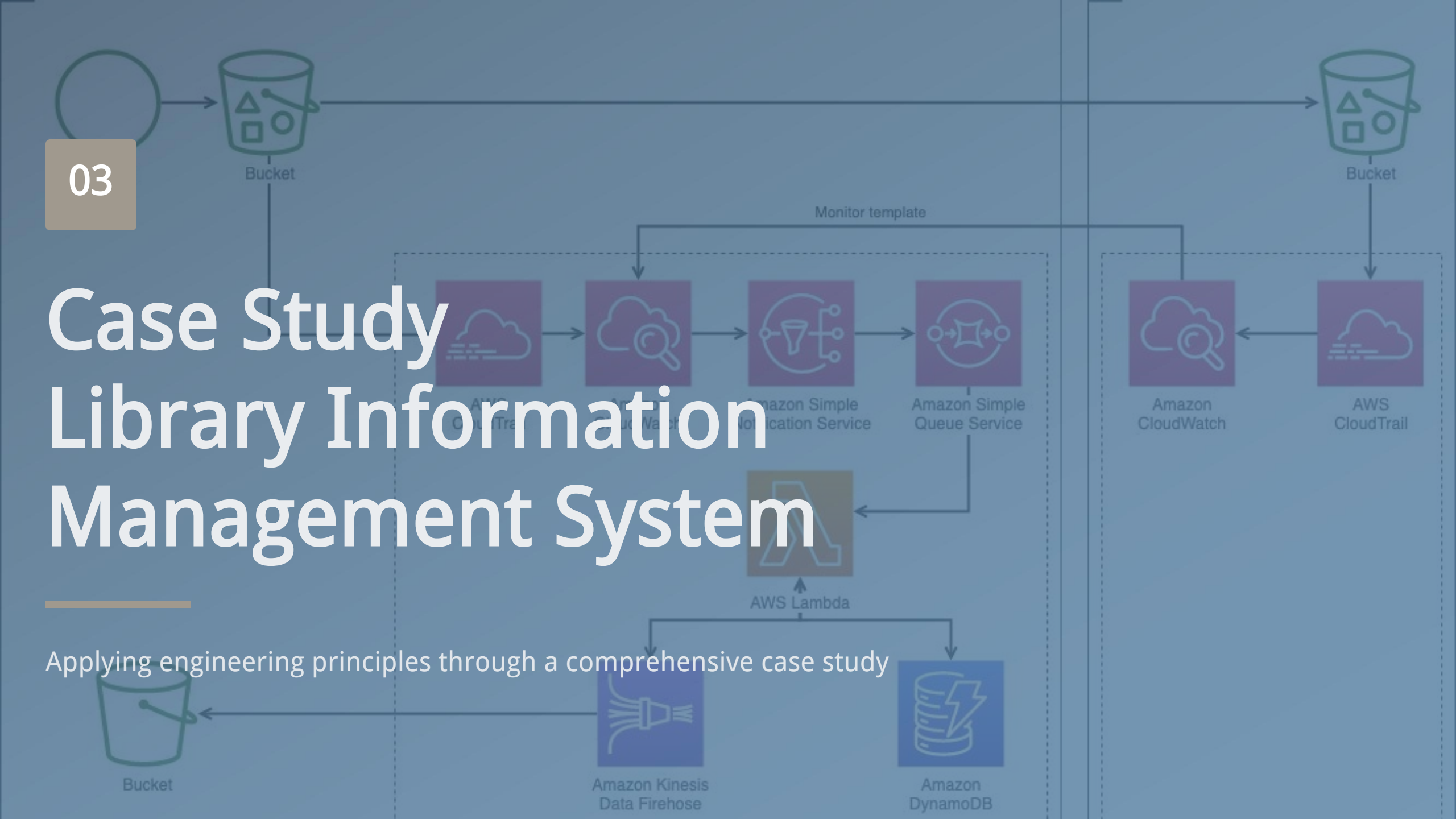
Such programs are completed by **project teams** and cannot rely on individual conception.

Documentation: Program maintenance requires detailed docs for knowledge transfer.

03

Case Study Library Information Management System

Applying engineering principles through a comprehensive case study



LIBRARY MANAGEMENT SYSTEM

Requirements Analysis

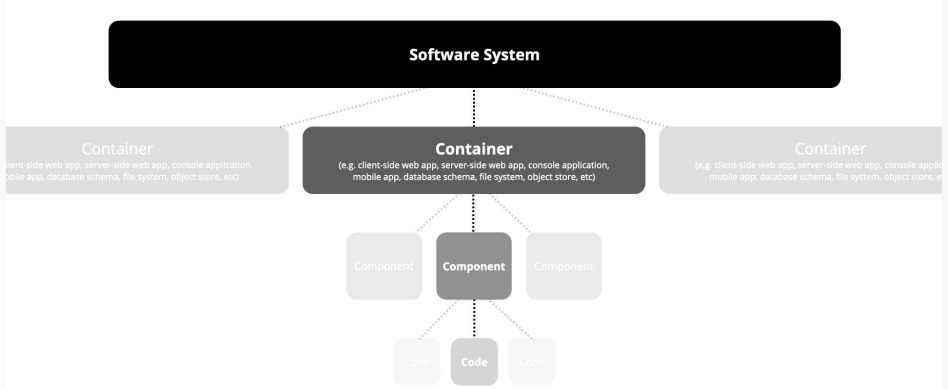
Understanding User Needs

The **purpose of program construction** is to meet user needs. The basis of program design is user requirements. To accurately grasp user needs:

1. **Communicate with Users**
   Establish communication to understand needs, workflows, and constraints.
2. **Establish Requirements Document**
   Create formal documentation capturing all identified requirements.
3. **User-Involved Review**
   Have users review and approve requirements to ensure accuracy.

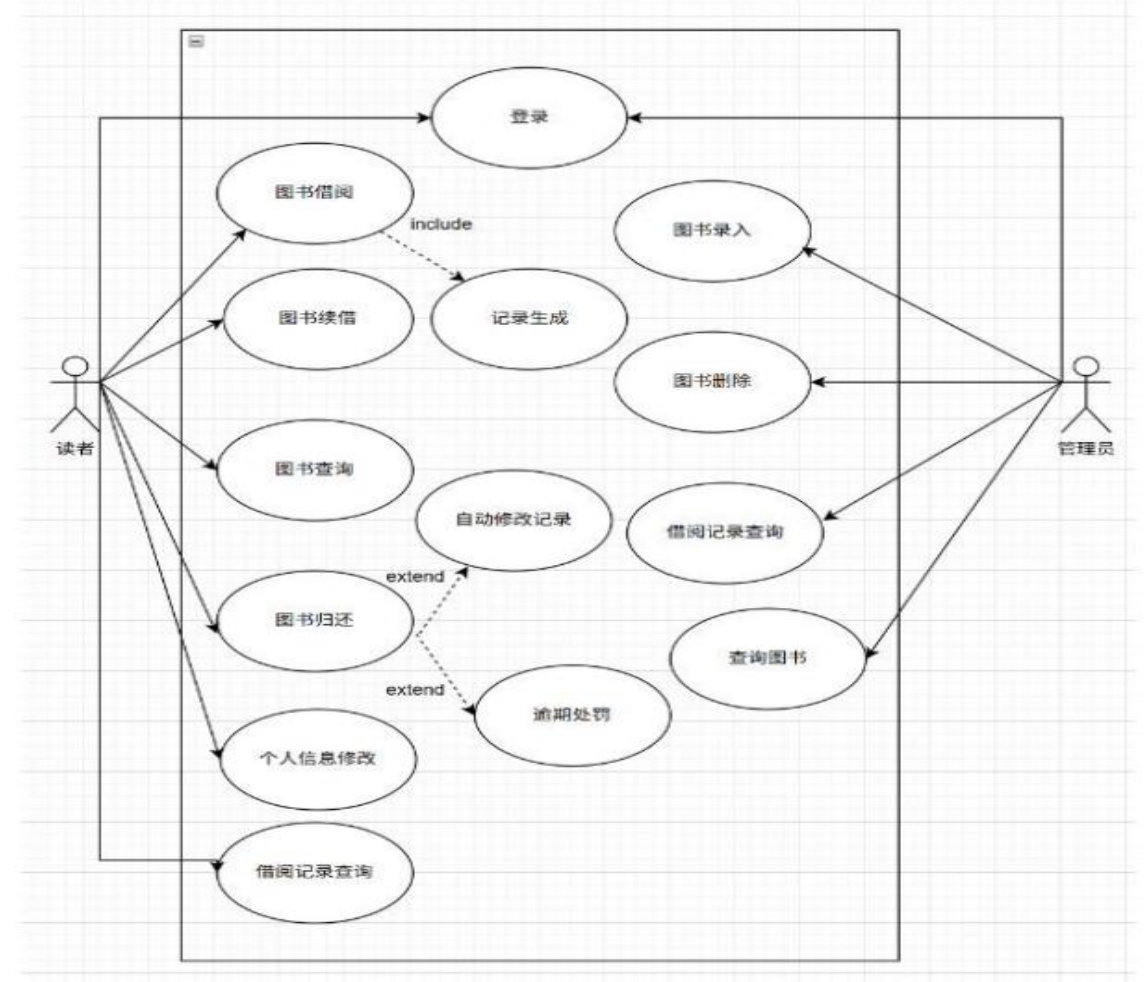
Key Library Requirements

- Book management
- Member management
- Check-out/in
- Search & query



A **software system** is made up of one or more **containers** (applications and data stores), each of which contains one or more **components**, which in turn are implemented by one or more **code** elements (classes, interfaces, objects, functions, etc).

Functional Design



Design Based on Requirements

The **basis of design** is the reviewed requirements specification. Before building, complete design work first.

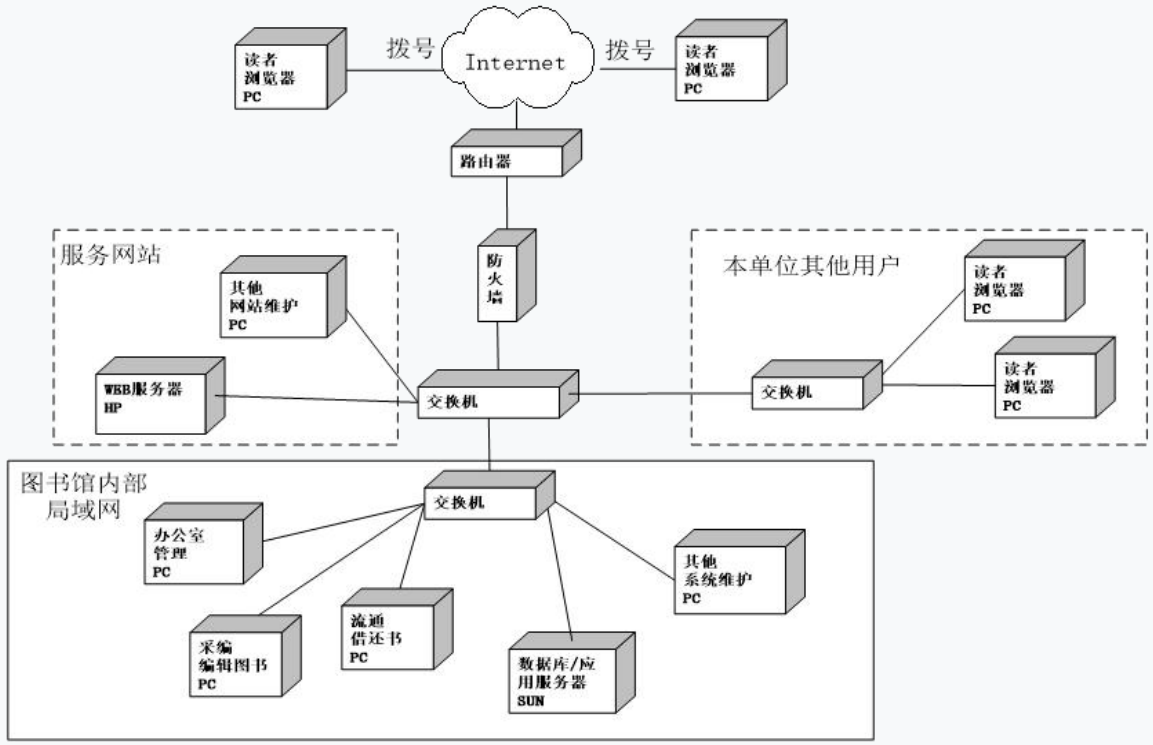
Design Definition

Design is a **logical construction** in a reviewable manner, starting at a high abstraction level.

Design Components

- **System Architecture Design**
  High-level system structure and component relationships
- **Functional Design**
  What the system should do
- **Data Structure Design**
  Information organization and storage
- **Deployment Design**
  Physical infrastructure and configuration

Physical Model Design



Physical Implementation

Physical model design translates the logical design into a concrete implementation plan, considering hardware, deployment, and storage.

Hardware Requirements

Server specifications, client machines, network equipment, and capacity planning for system load.

System Deployment

Network topology, server clustering, load balancing, and failover strategies.

Physical Storage

Database storage solutions, backup systems, archival strategies, and disaster recovery planning.

LIBRARY MANAGEMENT SYSTEM

Logical Model Design

System Structure

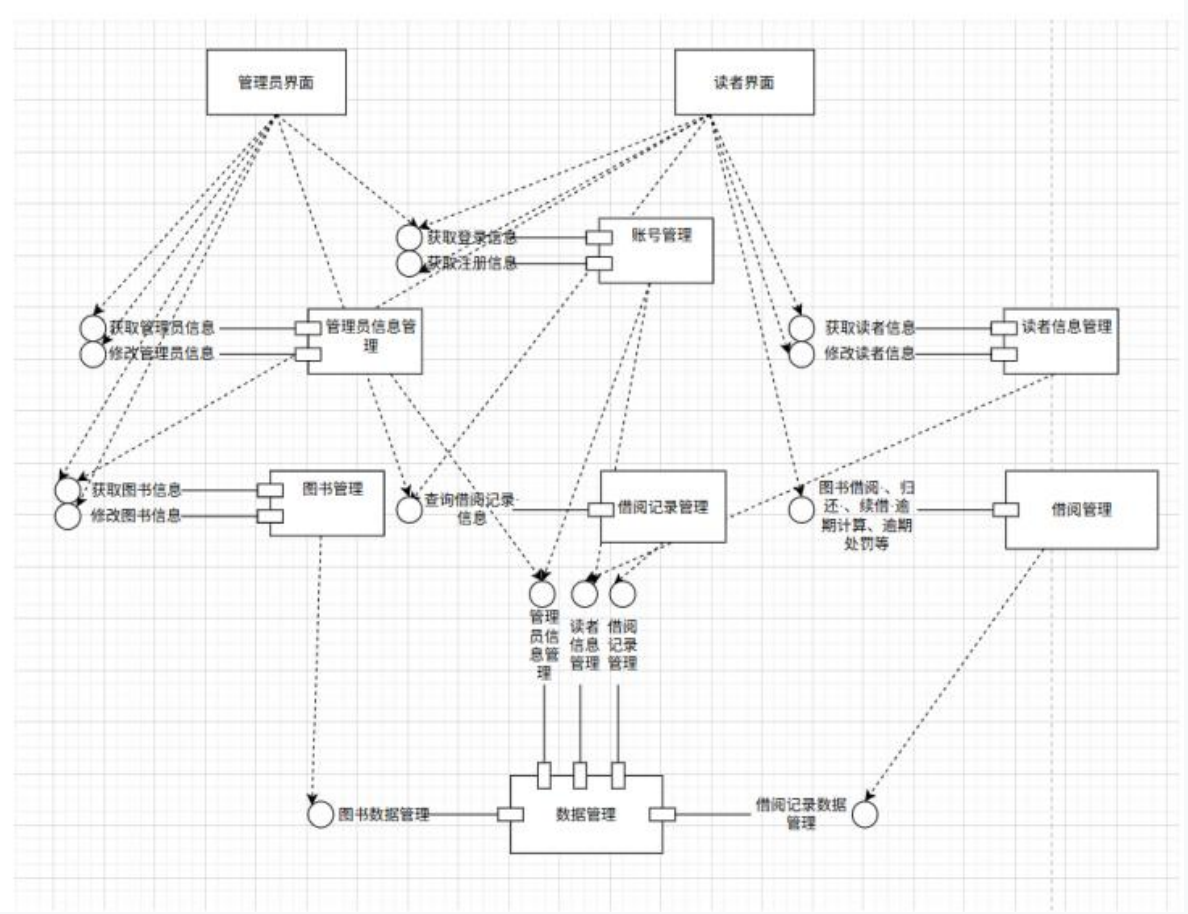
Logical model design defines the system's structure: data relationships, business rules, and processing logic, independent of physical implementation.

Design Focus

Focuses on **what the system does** rather than how it does it, remaining independent of technology choices.

Key Elements

- **Data Relationships**
  Entity relationships, cardinality
- **Business Rules**
  Constraints and validation logic
- **Processing Logic**
  Workflows and algorithms



LIBRARY MANAGEMENT SYSTEM

Data Design

Database & Structure

Data design focuses on database structure, entities, relationships, and integrity constraints.

Library System Data Entities

Book Entity

- ISBN (Primary Key)
- Title, Author, Publisher
- Publication Year

Member Entity

- Member ID (Primary Key)
- Name, Address, Phone
- Membership Type

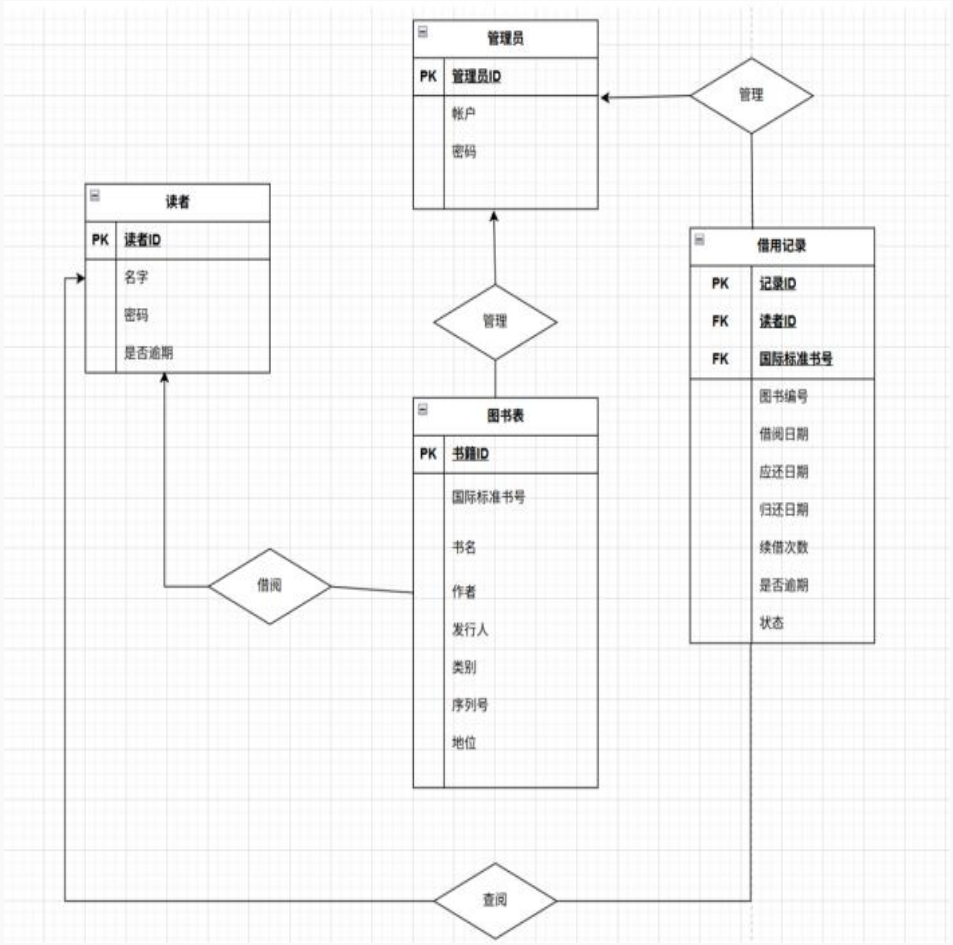
Loan Entity

- Loan ID (Primary Key)
- ISBN (Foreign Key)
- Member ID (Foreign Key)

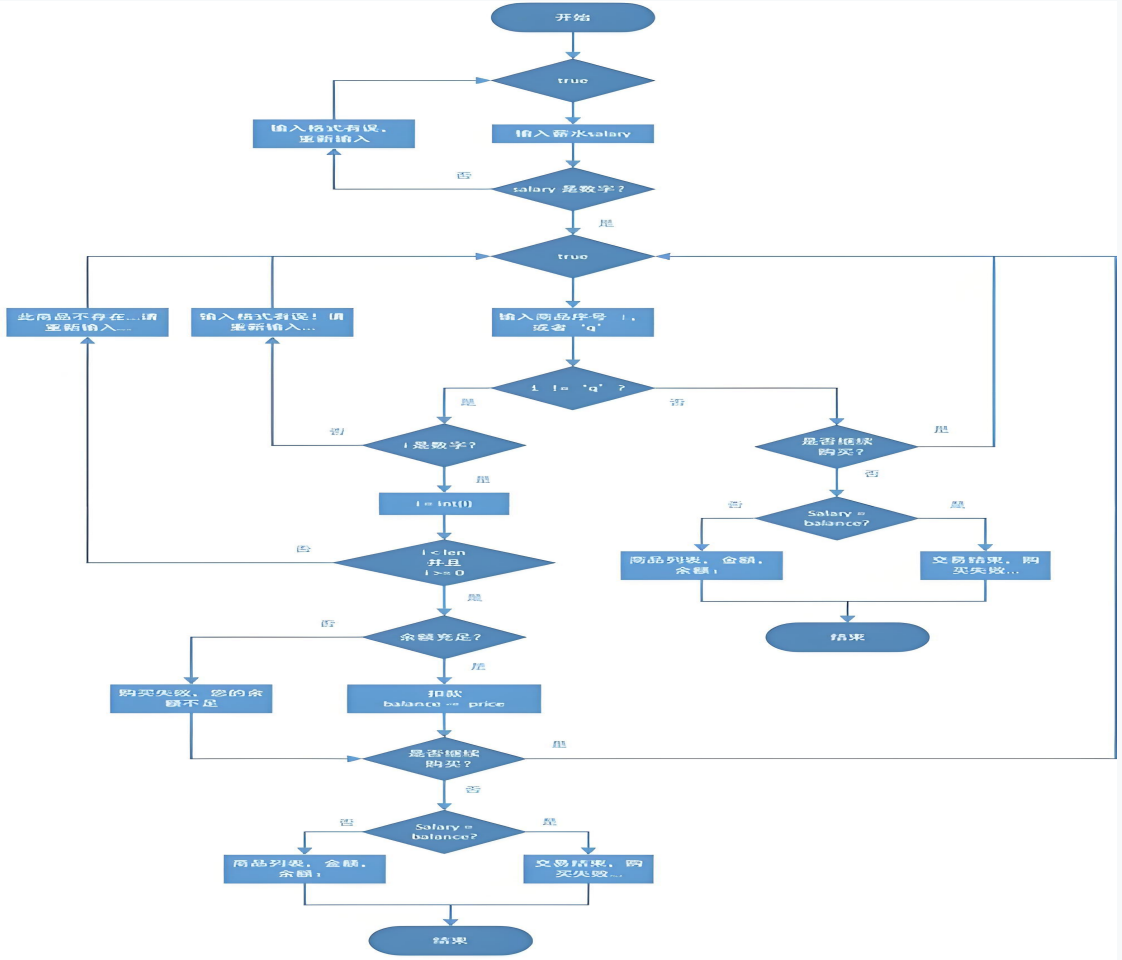
Staff Entity

- Staff ID (Primary Key)
- Name, Role, Password
- Contact Information

Design Goal: Create a normalized database that minimizes redundancy and maintains data integrity.



Process Design



Workflows & Logic

Process design defines operational workflows, algorithms, and procedural logic.

Core Library Processes

- **Book Acquisition:** New books → catalog → shelves
- **Member Registration:** Applications → approval → ID card
- **Book Check-out:** Validation → availability → record loan
- **Book Check-in:** Book return → condition check → update records
- **Overdue Handling:** Calculate fines → notifications → restrictions

Algorithm Focus: Each process needs clear step-by-step procedures with decision points.

LIBRARY MANAGEMENT SYSTEM

Interface Design

User Interaction

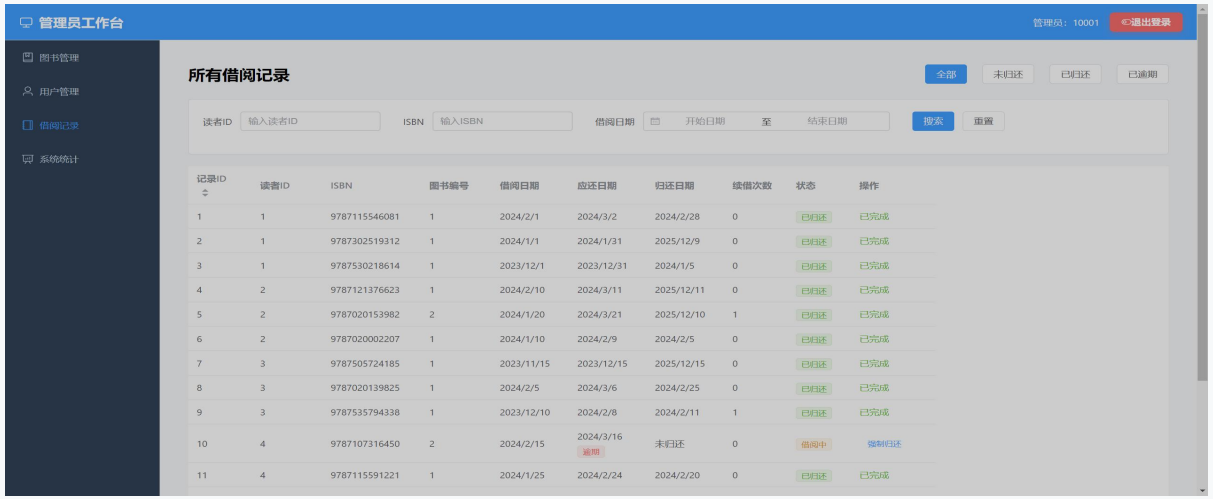
Interface design creates interaction patterns, screen layouts, and navigation flows to ensure usability.

Design Principles

- **User-Friendly**
  Intuitive operations
- **Efficiency**
  Minimize clicks
- **Accessibility**
  Support for disabilities

Library System Interfaces

- Login
- Search
- Book Mgmt
- Reports





ENGINEERING PRINCIPLES

Software Development as Engineering Process

Software Production = Engineering

Software production requires engineering, scaling, automation, and standardization, just like other industries (e.g., construction).

Intermediate Stages

Designing models, interfaces, data, processes. Quality checks at any time.

Efficient Tools

Compilers, debuggers, testing frameworks, project management systems.

Specialized Roles

Analysts, designers, programmers, testers, operators, maintainers.

Handcraft Inadequacy

Low-level workshops cannot meet large-scale development needs.

Engineering Characteristics

- ✓ Standardization
- ✓ Scalability
- ✓ Automation
- ✓ Quality Control

Solution

Software Engineering Development & Maintenance

EDUCATIONAL GOALS

Course Learning Objectives

01

Master Fundamental Knowledge

Master the **basic knowledge and theories** of software engineering to gain a comprehensive understanding of the discipline. This includes understanding software development lifecycles, methodologies, quality assurance, and project management principles.

02

Understand Development Process

Become familiar with the **general process of software project development and maintenance**, including planning, requirements analysis, system design, implementation, testing, deployment, and post-delivery maintenance activities.

03

Master Technical Methods

Master the **main ideas and technical methods** for software requirements analysis, design, coding, and testing phases. Learn practical techniques such as UML modeling, design patterns, version control, and automated testing.

04

Apply Software Engineering Thinking

Through learning and **course projects**, truly apply and experience software engineering thinking. Transform understanding of software development from individual pure programming activities to **system analysis and design**—the engineering approach.

Outcome: Students will transition from "coding" to "engineering software solutions" through systematic approaches.

LEARNING RESOURCES

Textbooks & References

Primary Textbook

Core course material

Software Engineering: A Practitioner's Approach

Comprehensive coverage of software engineering principles, methodologies, and practices.

- ✓ Lifecycle models and processes
- ✓ Requirements and design
- ✓ Testing and quality assurance
- ✓ Project management

Supplementary Texts

- Object-Oriented Software Engineering
- Software Engineering Theory and Practice
- Agile Software Development Principles

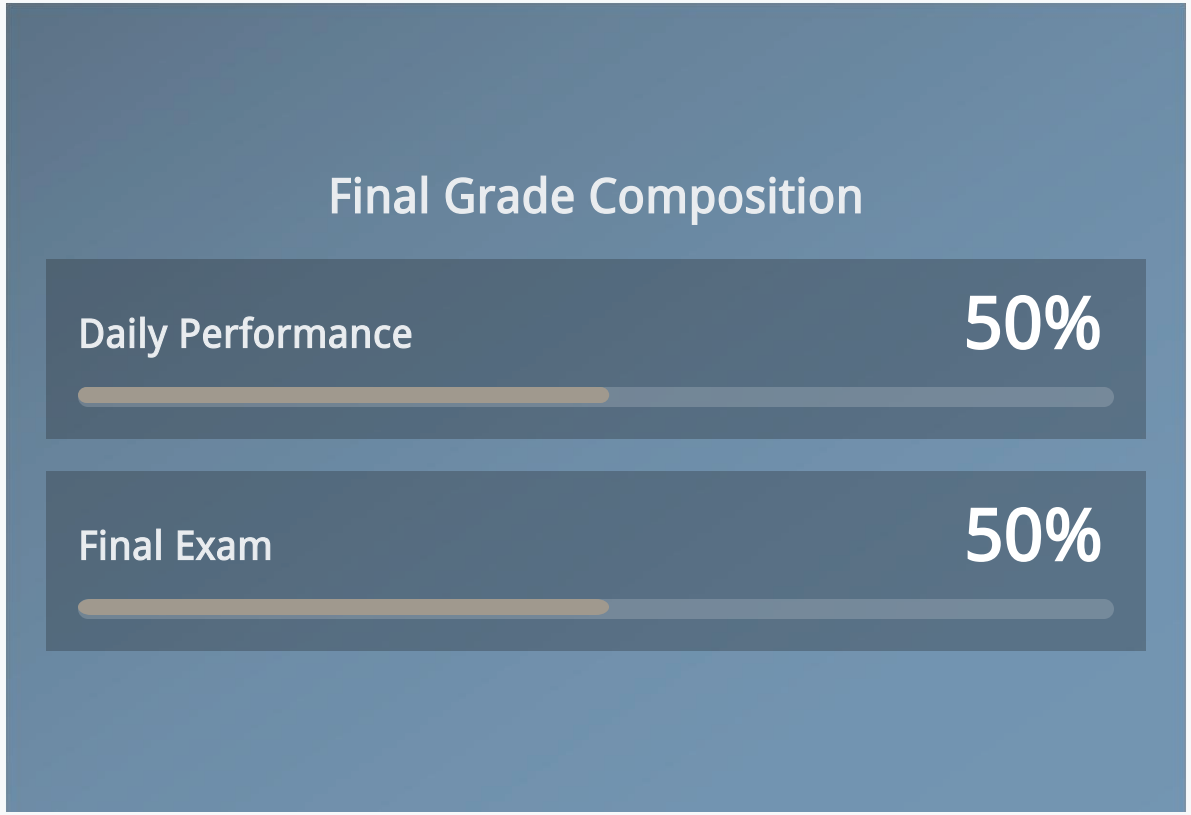
Online Resources

- IEEE Computer Society Digital Library
- ACM Digital Library
- Software Engineering Institute (SEI)

Note: Multiple references ensure comprehensive coverage. Students should use primary textbook alongside supplementary resources.

GRADING POLICY

Course Assessment



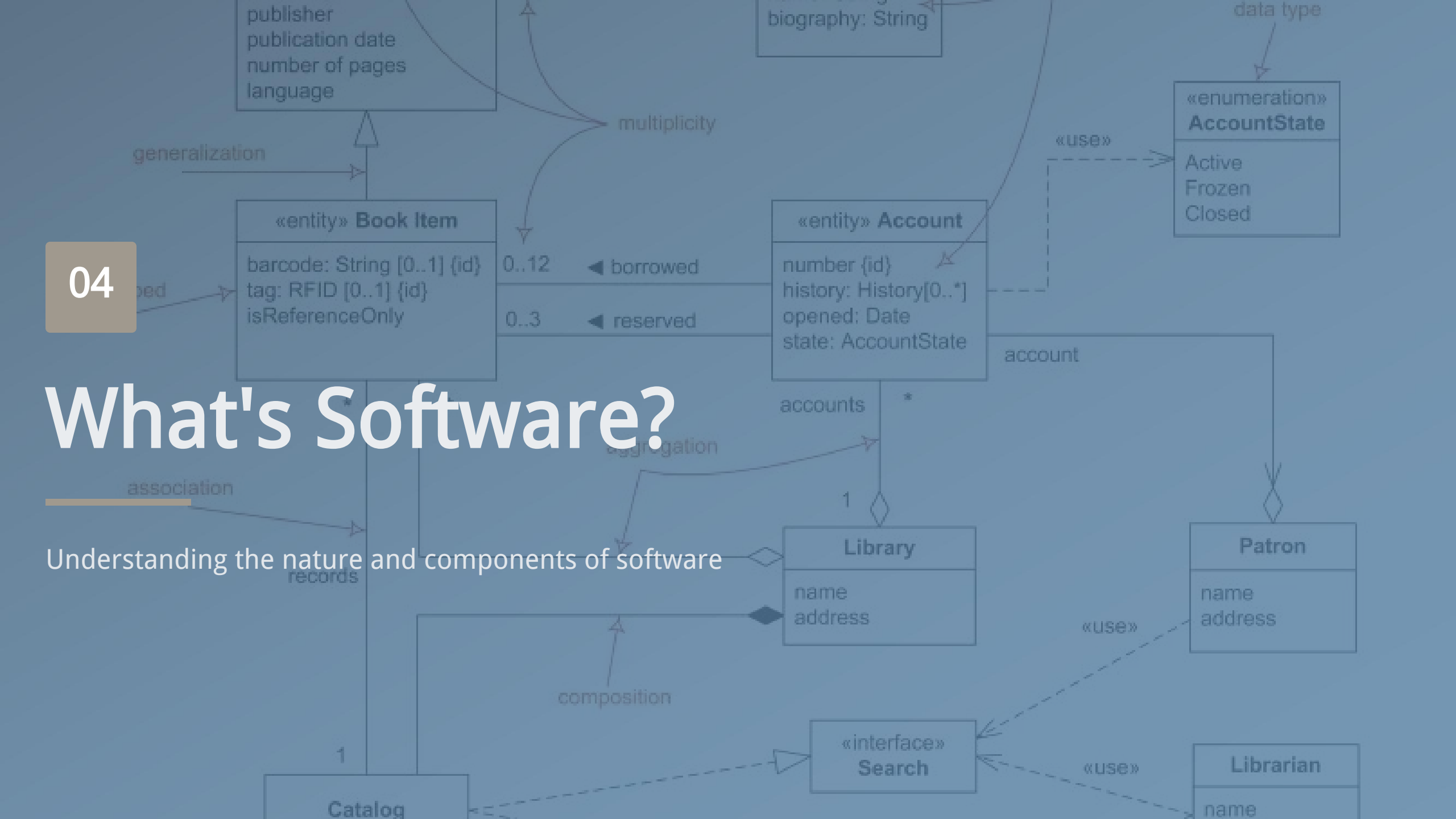
Daily Performance (50%)

Attendance	30%
Homework & Projects	30%
Midterm Exam	40%

Final Exam (50%)

Closed-book written examination covering all course materials.

- ✓ Theoretical concepts and principles
- ✓ Problem-solving and analysis
- ✓ Case study evaluation



04

What's Software?

Understanding the nature and components of software

SOFTWARE FUNDAMENTALS

Software Definition

What is Software?

Software is a set of items or objects that form a "configuration" that includes:

Programs	Documents	Data

Key Insight: Software is more than just code. It is a comprehensive configuration of interrelated components.



SOFTWARE COMPONENTS

Three Essential Components



Computer Programs (Instructions)

When executed, provide **desired features, function, and performance**. Programs are sequences of instructions that perform specific tasks, manipulate data, and deliver functionality to users.



Data Structures

Enable programs to **adequately manipulate information**. Data structures organize and store data efficiently, including files, databases, and in-memory structures that programs operate on.



Documentation (Descriptive Information)

Describes the **operation and use of programs**. Documentation includes user manuals, technical specifications, design documents, and code comments essential for understanding, using, and maintaining the software.

Programs
Executable instructions

+ Data
Information structures

+ Documents
Descriptive materials

DOCUMENTATION IMPORTANCE

Purpose of Software Documentation

Why Documentation Matters

Documentation serves multiple critical purposes throughout the software lifecycle, from development to maintenance.

Improve Visibility

Makes the software process visible to stakeholders.

Develop Efficiently

Provides clear guidance, reducing errors and rework.

Record Progress

Documents outcomes at the end of each period.

Technical Information

Records technical decisions during development.

Operation & Maintenance

Provides info on operation, maintenance, and training.

User Understanding

Helps users understand software functions.

Key Principle: Documentation is not optional. It is an integral part of software engineering that ensures long-term success.

DOCUMENTATION TYPES

Types of Software Documentation

1. **Feasibility Study Report**
   Economic and technical project feasibility analysis.
2. **Software Requirement Specification**
   Functional and non-functional requirements definition.
3. **Data Requirement Specification**
   Data needs and structure identification.
4. **Project Development Plan**
   Schedule, resources, and milestones.
5. **Testing Plan**
   Testing strategy and test cases.
6. **Conceptual Design Document**
   High-level architecture and component design.
7. **Detailed Design Document**
   Module internals and algorithms.
8. **Database Design Document**
   Database schema and constraints.
9. **Module Development Files**
   Individual module specifications.
10. **User Manual**
    End-user operation instructions.
11. **Software Instruction Manual**
    Detailed system functionality guide.
12. **Testing Analysis Report**
    Test results and quality assessment.
13. **Development Monthly Report**
    Regular progress tracking and issue documentation.
14. **Project Summary Report**
    Final project outcomes and lessons learned.

Lifecycle Coverage: Documentation spans the entire project, from conception to deployment and maintenance.



05

What's Good Software?

Quality considerations in software engineering

Quality Perspectives

Quality Strategy Required

Good software engineering must always include a strategy for producing quality software.



The Quality of the Product

Focuses on the **software product itself**—its correctness, reliability, performance, and usability. This perspective examines whether the software meets technical specifications and delivers expected functionality.



The Quality of the Process

Examines the **development and maintenance process**. A quality process ensures consistency, predictability, and continuous improvement, leading to fewer defects and better outcomes.



Quality in Business Context

Considers **business value** alongside technical quality. Software must deliver ROI, meet market needs, and align with organizational goals to be truly successful.

Integration: All three perspectives are equally important.

Product Quality

Different Perspectives

Users' Perspective

Users judge **external characteristics** they can observe:

- ✓ Correct functionality
- ✓ Number of failures
- ✓ Type of failures
- ✓ Performance & usability

Stakeholder Differences

Different stakeholders have different criteria for quality. A developer may prioritize elegant code, while a user cares about ease of use.

Developers' Perspective

Designers judge **internal characteristics**:

- ✓ Code quality & structure
- ✓ Types of faults
- ✓ Maintainability

Quality Models Needed

We need **quality models** to relate the user's external view to the developer's internal view, bridging perspectives.

Models translate user needs into technical requirements.

QUALITY FRAMEWORKS

Quality Models

Evolution of Quality Models

Quality models have evolved over decades to provide frameworks for evaluating and improving software quality.

Jim McCall (1977)

Early quality model organizing factors into three categories.

Barry W. Boehm (1978)

Quality model emphasizing utility and maintainability.

FURPS/FURPS+

Functionality, Usability, Reliability, Performance, Supportability.

R. Geoff Dromey

Model linking high-level quality attributes to low-level attributes.

ISO/IEC 9126 (1993)

International standard for quality characteristics.

ISO/IEC 25010 (2011)

Updated standard expanding quality model to include security and compatibility.

Evolution: Models have evolved from simple attribute lists to comprehensive frameworks.

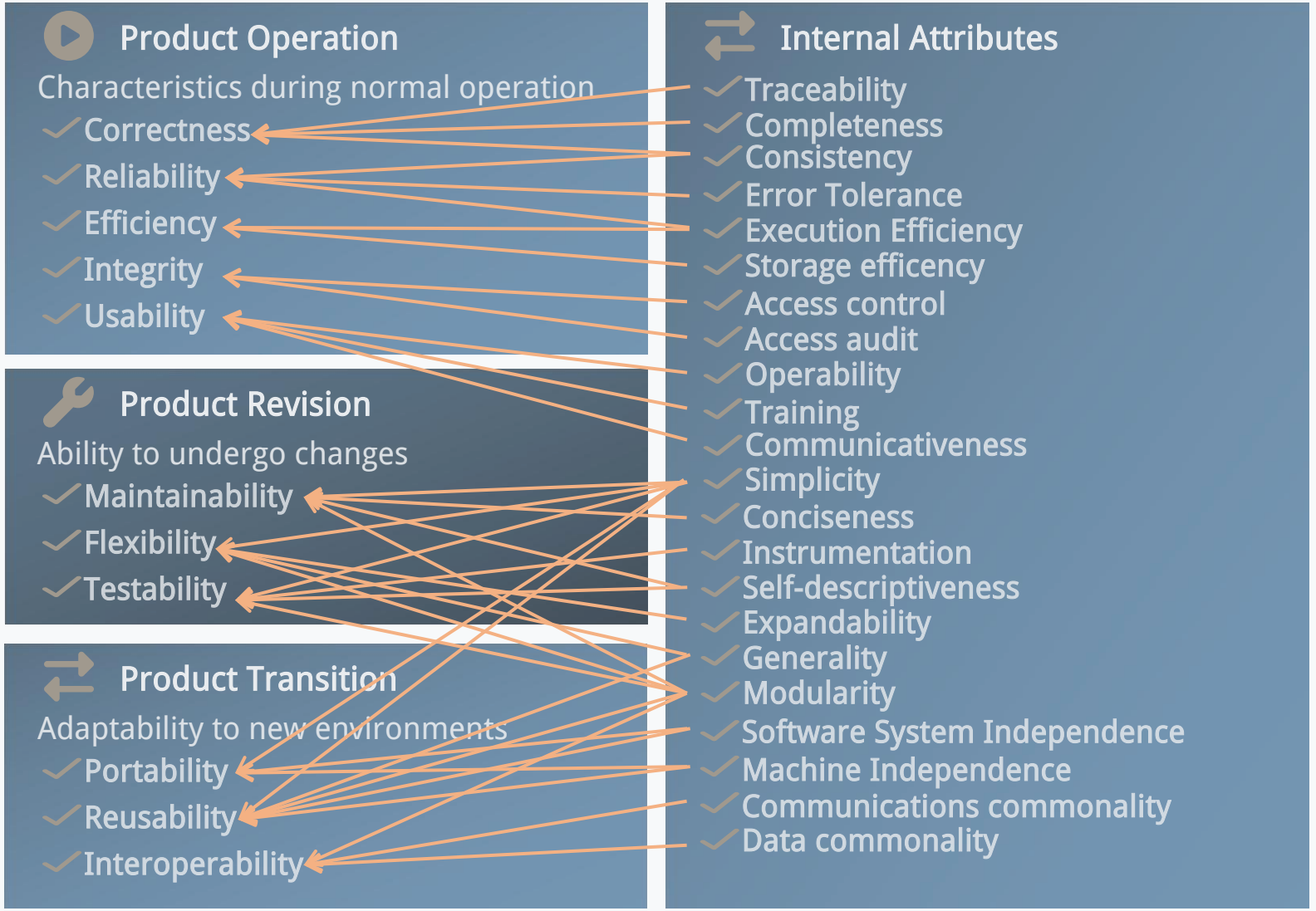
MCCALL'S MODEL

McCall's Quality Model

1、Quality Factors (High Level): These are the external quality attributes that are directly visible to users and managers. McCall defined 11 key factors.

2、Quality Criteria (Middle Level): These are internal attributes of the software that influence the high-level factors. They are more technical and often assessed by developers during the development process. Examples include simplicity, consistency, and self-descriptiveness.

3、Metrics (Low Level): These are quantitative or qualitative measures used to objectively assess the quality criteria, making the evaluation process concrete.



PROCESS QUALITY

Quality of the Process

Process Importance

Quality of the **development and maintenance process** is as important as the product quality.



Why Model the Process?

The development process needs to be modeled to address critical questions:

- Where to find particular kinds of faults?
- How to find faults early?
- How to build in fault tolerance?
- What are alternative activities?

Fault Detection

Identify where faults are likely to occur

Early Detection

Find faults as early as possible

Continuous Improvement

Optimize development practices

PROCESS IMPROVEMENT

Process Improvement Models



SEI's Capability Maturity Model (CMM)

The **Capability Maturity Model** defines five maturity levels for software processes, from initial (ad-hoc) to optimizing (continuous improvement). It provides a roadmap for organizations to improve their development capabilities.



ISO 9000

ISO 9000 is an international standard for quality management systems. It focuses on ensuring consistent processes, documentation, and customer satisfaction. Applicable to software as well as other industries.



SPICE (ISO/IEC 15504)

Software Process Improvement and Capability Determination (SPICE), now ISO/IEC 15504, is an international framework for assessing process capability. It provides a structured approach for evaluating and improving development processes.

Goal: These models help organizations systematically improve processes, reduce defects, and increase efficiency.

BUSINESS VALUE

Quality in Business Context

Beyond Technical Quality

Business value is as important as technical value.



Quantifying Business Value

Business value (in relationship to technical value) must be quantified. A common approach:

Return on Investment (ROI)

ROI measures the gain relative to cost: **ROI = (Gain - Cost) / Cost × 100%**

Cost Reduction

Lower development and maintenance costs

Revenue Generation

Increased sales and market share

Risk Mitigation

Reduced project failure risk

SOFTWARE ROLES

Software's Dual Roles

Software is Product

Software **delivers computing potential** and produces, manages, acquires, modifies, displays, or transmits information.

- ✓ Direct end-user functionality
- ✓ Information processing capabilities
- ✓ Data management and storage
- ✓ User interface and interaction

Vehicle for Delivering Product

Software **supports or provides system functionality** and enables other capabilities.

- ✓ Controls other programs (e.g., OS)
- ✓ Enables communications
- ✓ Builds other software (tools)
- ✓ Provides development platforms

Software Provides Information

In both roles, software fundamentally **provides information** and enables information processing capabilities.

06

Software Characteristics

Unique properties that distinguish software from traditional products

UNIQUE PROPERTIES

Software vs Hardware

</>Engineered, Not Manufactured

Software is **engineered or developed**, not manufactured in the traditional sense. It is a **logical product**, not a physical one.

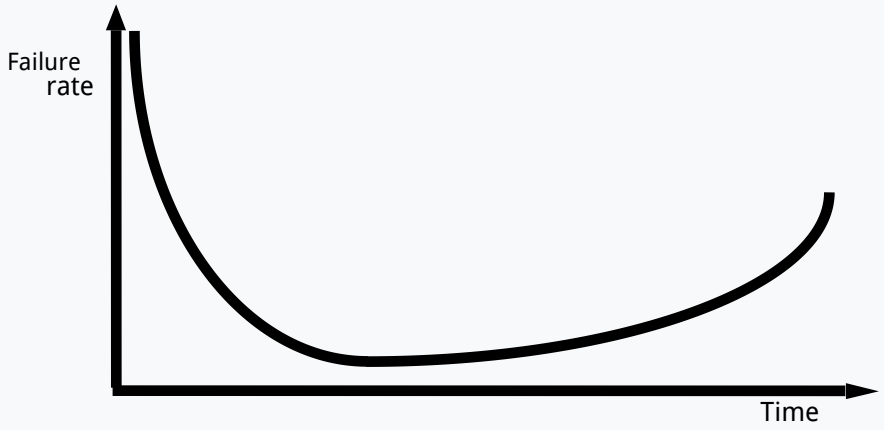
∞No Wear and Tear

Software **does not wear out**. Its failure rate curve differs significantly from hardware.

Failure Rate Comparison

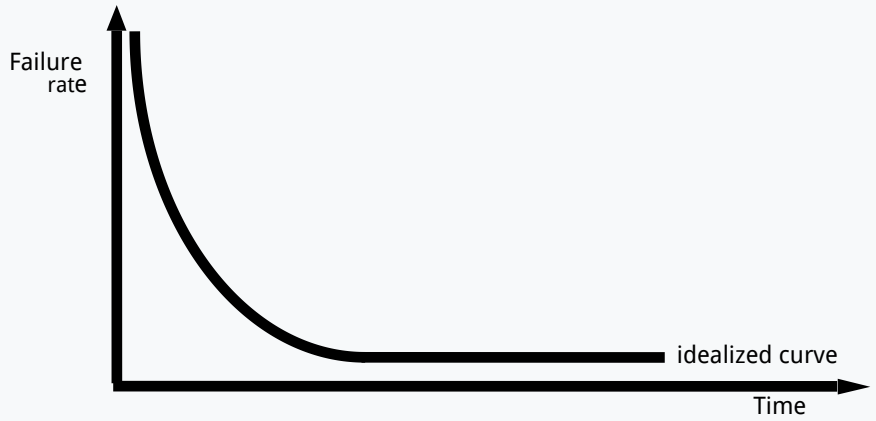
Hardware (Idealized)

> **"Bathtub Curve":** High initial failures (burn-in), then stable (useful life), then rising (wear-out).



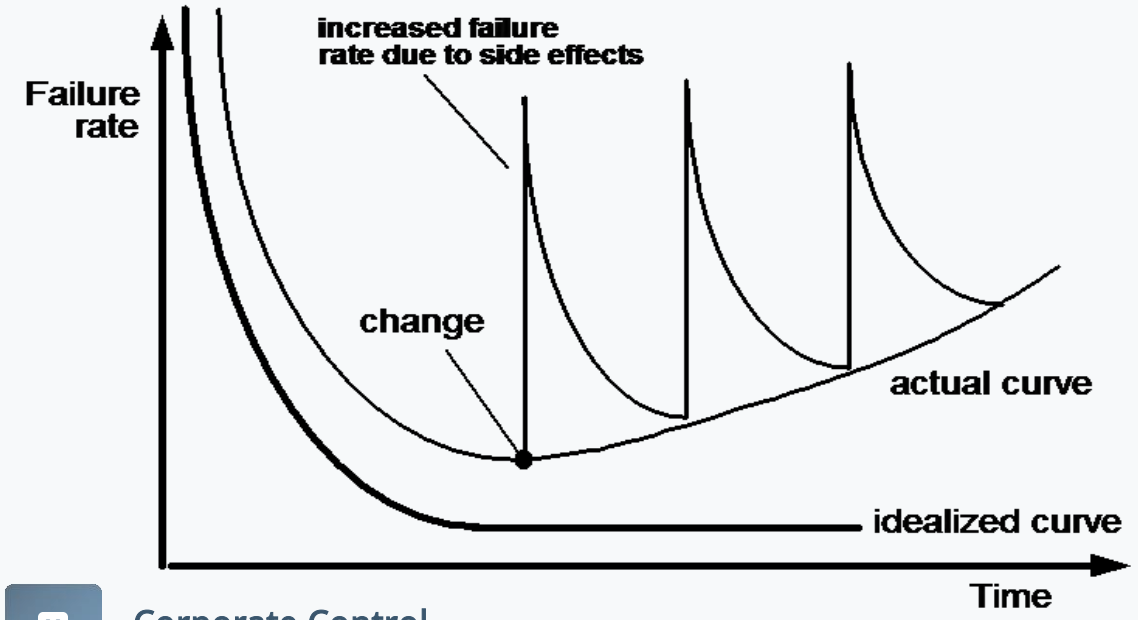
Software

> No physical wear. If defects are fixed, curve should **decrease** over time. However, **multiple changes** introduce new errors.



SOFTWARE AGING

Software Deterioration



Why Software Deteriorates

Software is **deteriorating** due to multiple factors inherent in its nature as a malleable product.

Corporate Control

Most serious corporations control and constrain changes to prevent deterioration.

Unclear Requirements

Most software is custom built, and customer never really knows what they want.

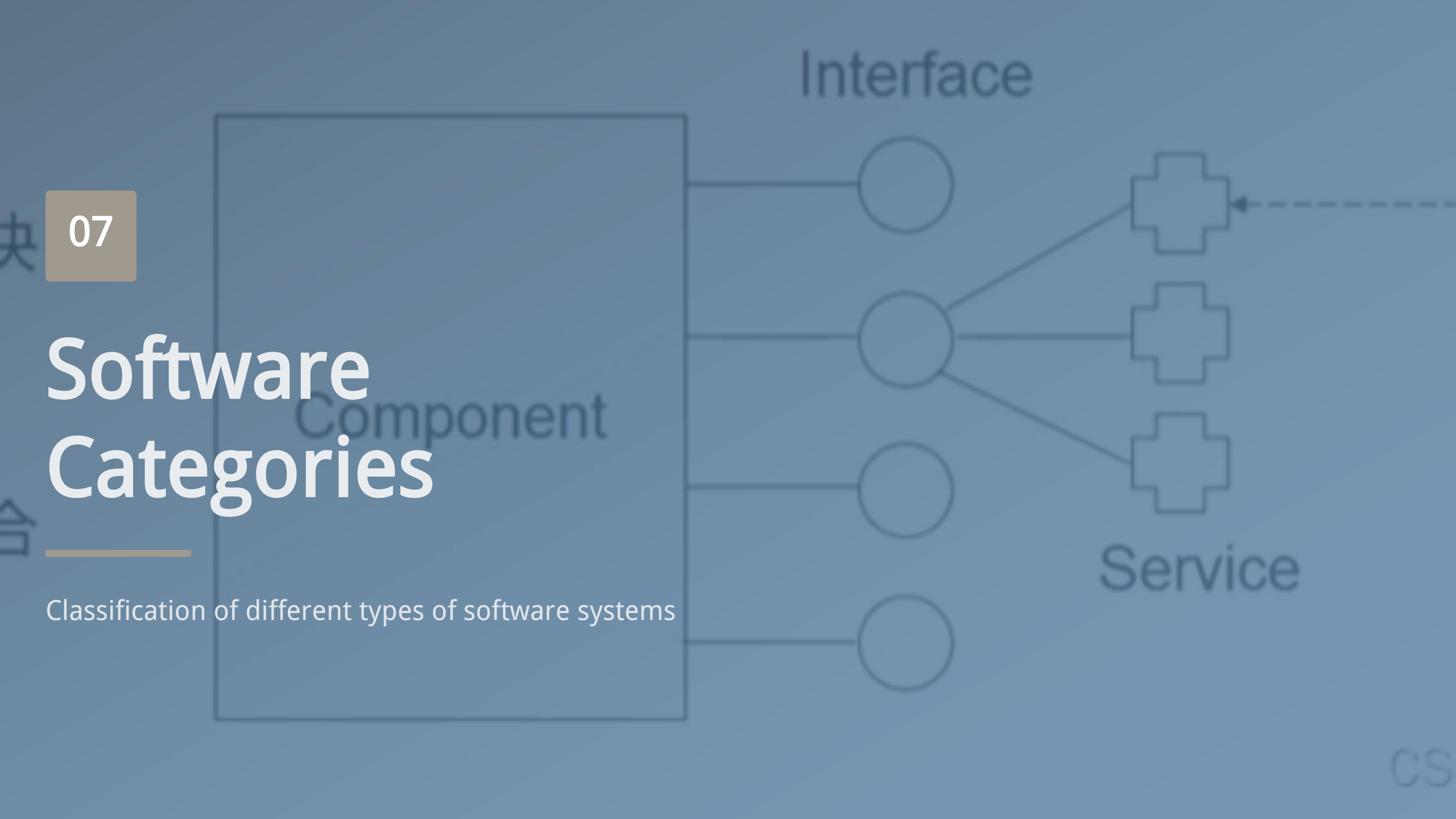
Multiple Change Requests

Multiple change requests introduce errors in component interactions, causing deterioration.

07

Software Categories

Classification of different types of software systems



SOFTWARE TYPES

System Software

Definition

System software serves other programs, providing foundational services and infrastructure.



Development Tools

- ✓ **Compilers**
  Translate high-level code to machine code
- ✓ **Editors**
  Text editing for code and documentation
- ✓ **File Management**
  Organize and manipulate files
- ✓ **Debuggers**
  Find and fix program errors

System Infrastructure

- ✓ **Operating System Components**
  Memory, process, file system managers
- ✓ **Device Drivers**
  Hardware device control and communication
- ✓ **Networking Software**
  Protocol stacks, network management
- ✓ **System Utilities**
  Disk cleanup, backup, security tools

SOFTWARE TYPES

Application Software

Business-Focused Solutions

Application software consists of standalone programs that solve a specific business need or problem.

Point-of-Sale (POS)

Transaction processing systems used in retail and hospitality.

- Process sales transactions
- Manage inventory in real-time
- Generate business reports

Flight Booking System

Reservation systems for airlines and travel agencies.

- Search and book flights
- Manage passenger information
- Process payments and ticketing

Characteristics: Application software is typically domain-specific, with features tailored to business requirements.

SOFTWARE TYPES

Engineering/Scientific Software

Computation & Analysis

Engineering/Scientific software focuses on numerical computing and technical problem-solving.



Scientific Computing

- ✓ **Numerical Computing**
  Complex calculations in physics, chemistry, biology
- ✓ **Data Analysis**
  Statistical processing and visualization
- ✓ **Research Simulation**
  Model complex systems and phenomena

Engineering Applications

- ✓ **Computer-Aided Design (CAD)**
  Design and modeling of engineering components
- ✓ **System Simulation**
  Simulate behavior before physical prototyping
- ✓ **Interactive Applications**
  Real-time visualization and manipulation

SOFTWARE TYPES

Embedded Software

Embedded Systems

Embedded software resides in devices, controlling their functions. It is typically highly specialized and optimized for specific hardware.

Key Characteristics

- ✓ Tight hardware integration
- ✓ Resource constraints (memory, CPU)
- ✓ Real-time performance requirements
- ✓ High reliability and safety

Market Growth: Embedded systems are everywhere, from smart home devices to industrial control systems.

Home Appliances

Keypad control for home appliances demonstrates simple embedded systems.

- Microwave oven controllers
- Washing machine programs
- Smart thermostat systems

Automotive Systems

Modern automobiles contain **dozens of embedded systems** controlling various functions.

- Fuel injection and engine control
- Anti-lock braking systems (ABS)
- Airbag deployment systems

SOFTWARE TYPES

Product-line Software & Web Applications

Product-line Software

Designed to provide a **specific capability for use** by general consumers.

- Word Processing
- Spreadsheets
- Computer Graphics
- Multimedia
- Entertainment
- Database Mgmt

Web Applications

Web-based applications accessible through browsers, serving various purposes.

- E-commerce
- B2B Systems
- B2P Platforms
- Blogs
- Social Media
- Cloud Services

Market Characteristics

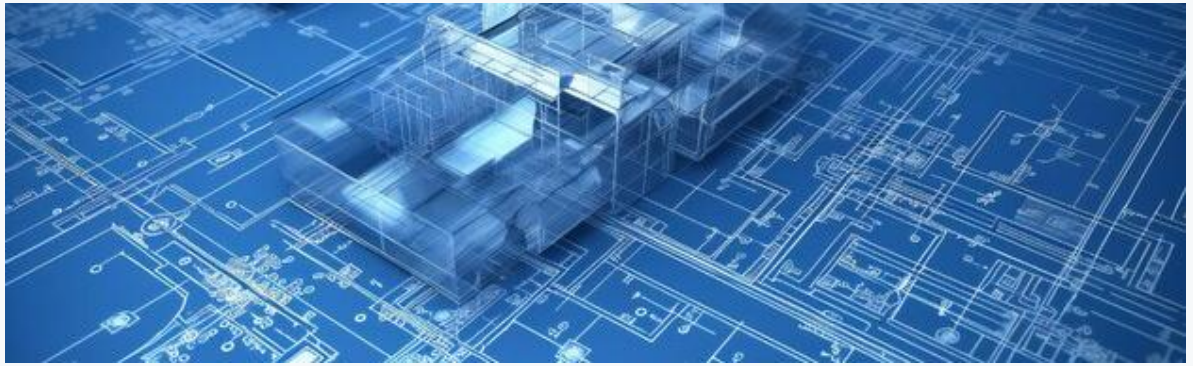
Both categories serve **broad markets** with large user bases. Success depends on usability, features, and ecosystem integration.

SOFTWARE TYPES

AI Software

Intelligence Simulation

AI software simulates human intelligence: learning, reasoning, problem-solving, perception, and language understanding.



Expert Systems

Solve complex problems using knowledge bases and inference engines.

Neural Networks

Pattern recognition, classification, and prediction tasks.

Game Playing

Strategic decision-making in competitive environments.

Evolution: AI software has evolved from rule-based systems to machine learning and deep learning approaches.

EMERGING TRENDS

New Software Categories



Ubiquitous Computing

Computing everywhere through wireless networking, 5G or Wi-Fi oriented computing.



Netsourcing

Web as a computing engine, delivering software as a service over the internet.



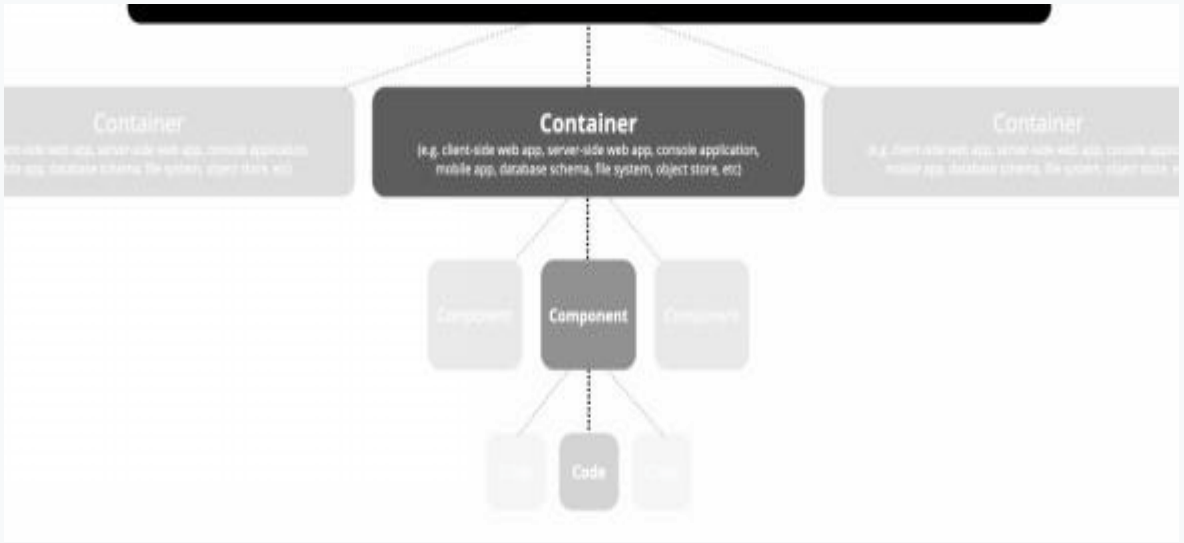
Open Source

"Free" source code open to the computing community, collaborative development.

Furthermore... The software landscape continues to evolve rapidly with new paradigms emerging.

EMERGING TECHNOLOGIES

Emerging Software Categories



Data Mining

Discovering patterns and insights from large datasets.

Cloud Computing

Delivering computing services over the internet.

Cognitive Machines

Systems that simulate human thought processes.

Software for Nanotechnologies

Software to design and control molecular-level devices.

Convergence: These emerging categories often overlap and converge, creating new opportunities and challenges.

LEGACY SYSTEMS

Legacy Software

Definition

Legacy software refers to systems that have been used for a long time and have become difficult to maintain, yet remain critical to business operations.

Why Must Legacy Software Change?

- Adapt to new computing environments or technology
- Enhance to implement new business requirements
- Extend to make it interoperable with modern systems
- Re-architect to make it viable within a network environment

Age Factor

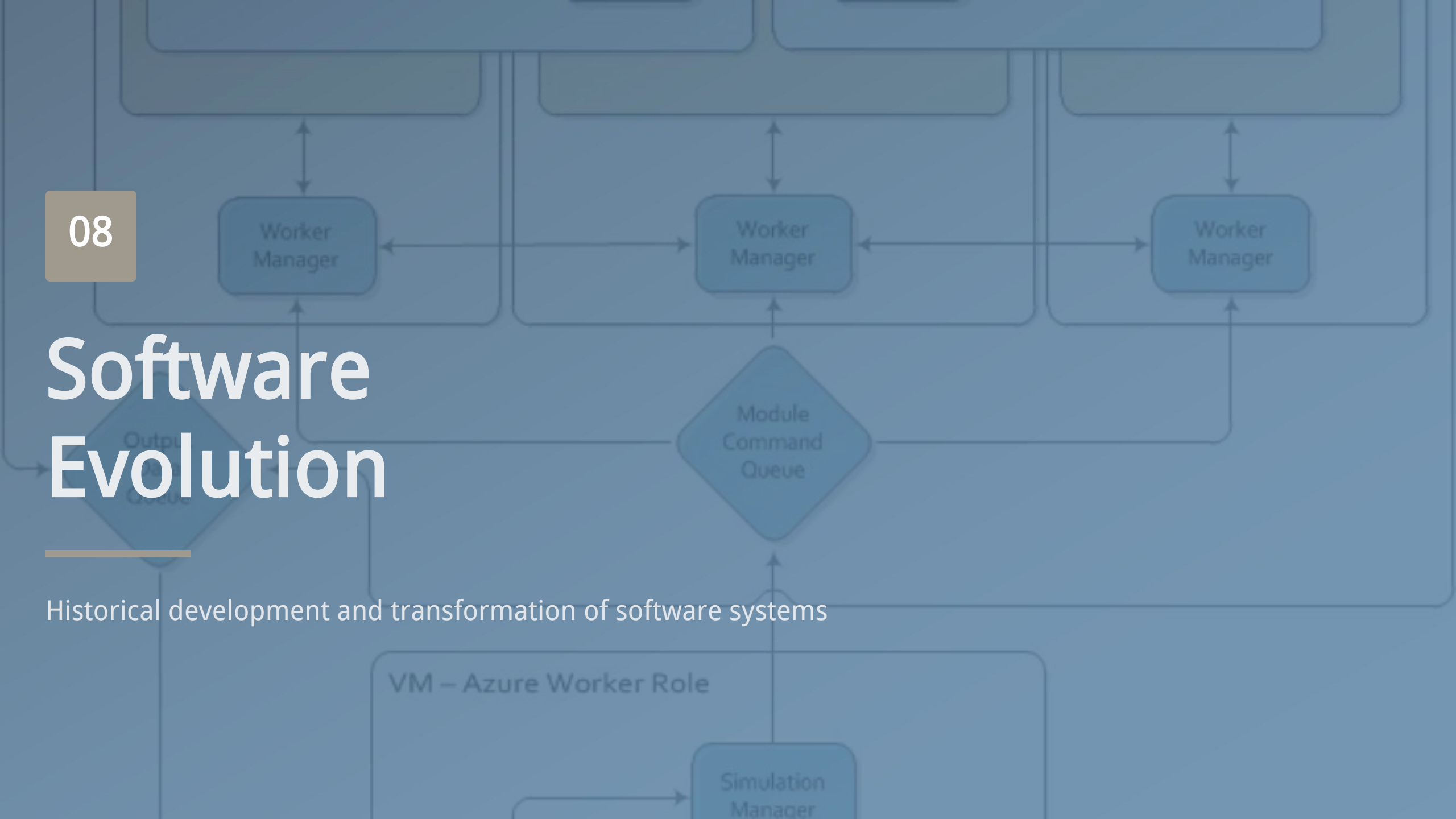
Often 10+ years old, built with outdated tech

Maintenance Challenges

Poor documentation, original developers gone

Business Risk

Cannot be replaced easily due to business dependency



08

Software Evolution

Historical development and transformation of software systems

HISTORICAL DEVELOPMENT

Software Evolution Phases

1. **Batch Processing Era**
   Custom software for mainframes, single-user systems with punched card input.
2. **Multi-user Systems**
   Time-sharing, real-time processing, introduction of databases and software products.
3. **Distributed Systems**
   Intelligence added, low-cost hardware, expert systems, object-oriented programming.
4. **Powerful Desktop Systems**
   OO systems, expert systems, neural networks, parallel computing, network computers.



SOFTWARE EVOLUTION LAWS

Lehman's Laws of Software Evolution

E-Type Programs

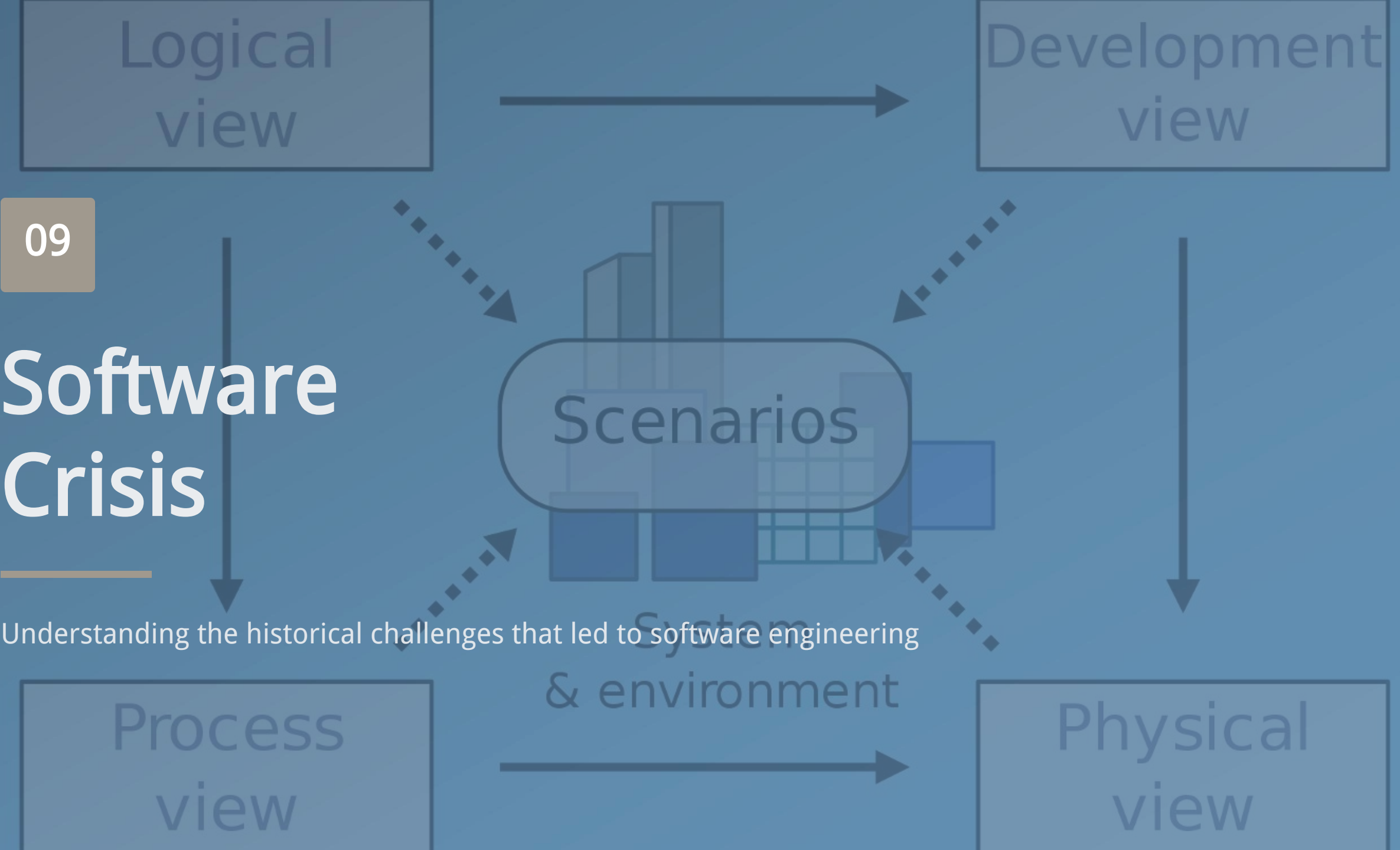
Based on decades of research, Lehman defined **E-type programs** (embedded in real-world domains) and proposed eight laws of software evolution.

1. **Continual Change:** A system used in real-world environment must keep changing; otherwise it becomes progressively less useful.
2. **Increasing Complexity:** Complexity rises as system evolves, unless special effort is spent to reduce it (e.g. refactoring).
3. **Self Regulation:** Software evolution process is self-regulating; metrics of each release follow stable statistical trends.
4. **Organizational Stability:** The effective development rate remains roughly constant over system lifetime, largely independent of added resources.
5. **Conservation of Familiarity:** Incremental change per release stays limited; too large changes break team's understanding of the system.
6. **Functional Growth:** System functionality must keep growing to maintain user satisfaction.
7. **Quality Decline:** System quality will decline unless it is carefully adapted to new operating-environment changes.
8. **Feedback System:** Software evolution is multi-agent multi-loop feedback system; improvements must consider feedback factors.

09

Software Crisis

Understanding the historical challenges that led to software engineering



THE CRISIS

What is Software Crisis?

Historical Context

The term **"software crisis"** has been used since the late 1960s to describe recurring system development problems that plagued the industry.

Budget Overruns

Projects running over-budget due to poor estimation and planning.

Schedule Delays

Projects running over-time, missing deadlines.

Inefficiency

Software was very inefficient, consuming excessive resources.

Low Quality

Software was of low quality with numerous defects.

Requirement Failure

Software often did not meet requirements.

Unmanageable

Projects were unmanageable and code difficult to maintain.

Ultimate Failure

Software was never delivered — projects were abandoned after consuming significant resources.

REAL-WORLD EXAMPLES

Crisis Case Studies

Notable Failures

Real-world examples of software crisis demonstrate the magnitude of the problem and its economic impact.

California DMV System

California DMV spent **over $43 million** on a system to merge driver and vehicle registration. The system was abandoned without ever being used.

American Airlines Reservation System

A failed **$165 million effort** by American Airlines to link its flight-reservation software with Marriott, Hilton, and other hotel reservation systems.

Arthur Andersen Report (1989)

In March 1989, Arthur Andersen reported that more than **$300 billion per year** was spent on commercial software in the United States, but only 8% resulted in software that was delivered AND worked.

ROOT CAUSES

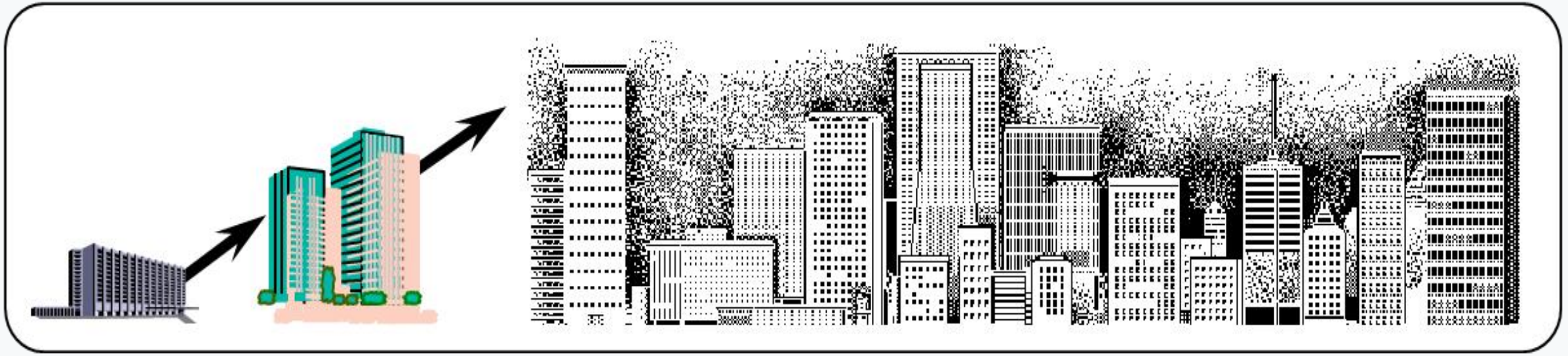
Crisis Causes: Rolling Baseline

⚖Unstable Requirements

Rolling baseline issues create instability throughout the project lifecycle, making it difficult to establish a stable foundation for development.

The Challenge

Shorter development cycles are business requirements, but **initial requirements are usually poorly defined**, creating a moving target.



ROOT CAUSES

Crisis Causes: Complexity

The Complexity Challenge

Software complexity is perhaps the most fundamental challenge facing developers. Unlike physical products, software complexity can grow exponentially.

Increasing Demands

Business software demands are increasing, requiring more features and capabilities.

No Complete Understanding

No one understands the entire system. Different teams work on different parts.

Knowledge Loss

Legacy systems must be maintained, but the original developers are gone.

ROOT CAUSES

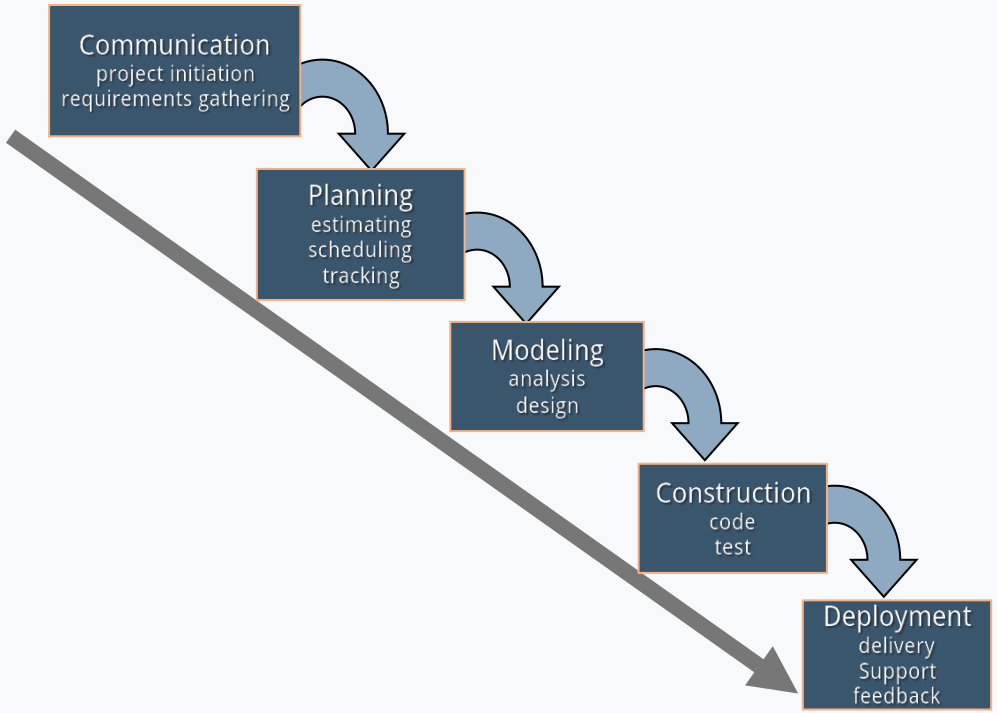
Crisis Causes: Risk Management Failure

⚠ Failure to Manage Risk

Inadequate risk management is a major contributor to software project failures. Risks must be identified, assessed, and mitigated throughout the project lifecycle.

The Waterfall Problem

- ✕ The waterfall lifecycle can delay problem identification
- ✕ No proof that the system will run until late in the lifecycle
- ✕ The result is maximum risk exposure



THE SOLUTION

Software Development as Engineering

An Engineering Project

Software development is an engineering project that requires systematic approaches, not ad-hoc coding.

Good Organization

Clear structure, defined roles, effective communication, and coordinated teamwork are essential for project success.

Rigorous Management

Systematic planning, monitoring, and control of projects, including risk management and quality assurance.

Multiple Developers

Collaboration among team members with complementary skills, coordinated through established processes.

Efficient Techniques

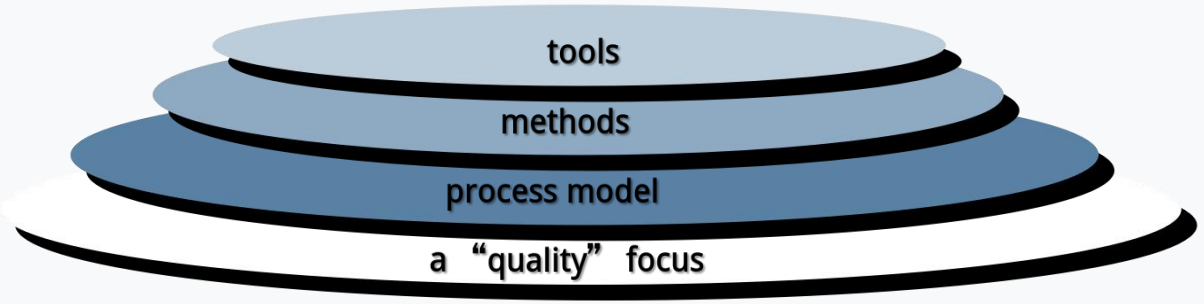
Appropriate methods, tools, and technologies that improve productivity and quality throughout the lifecycle.

THE SOLUTION

Solution: Software Engineering

The Answer

Solution is using **techniques (methods and tools)**, together with **organization and management** — Software Engineering.



🎓Definition

Software Engineering is an emerging discipline focused on studying how to well develop and maintain software from technical and management perspectives.

🔑Essence of SE

Max. Gain with Min. Cost!

Optimize value while minimizing resources

Transformation: Software engineering transformed software development from an art to an engineering discipline.

REAL-WORLD CHALLENGES

Bad Experiences: Requirements Issues

Common Requirements Problems

Even with software engineering practices, requirements-related challenges remain among the most persistent problems.

1. **Customer Unavailability**
   Customers are too busy to spend time working with analysts or developers on their requirements.

2. **False User Representatives**
   User surrogates such as marketing or managers claim to speak for the user, but they really don't.

3. **Everything is Critical**
   Customers claim all the requirements are critical, so they do not prioritize them.

4. **Ambiguity and Missing Information**
   Developers encounter ambiguities and missing information when coding, so they have to guess.



Lesson: Effective requirements engineering is critical to project success.

REAL-WORLD CHALLENGES

Bad Experiences: Scope Creep

Scope Management Challenges

Scope creep is the uncontrolled expansion of project scope without corresponding increases in resources or timeline. It is one of the most common causes of project failure.

Common Patterns

- ✓ Customers sign off on requirements but make continuous changes
- ✓ Project scope increases as requirements change
- ✓ Schedule slips because no more resources are provided

Feature Bloat

Functionality/features are requested but never used. Studies show that up to 50% of features are rarely or never used.

Customer Dissatisfaction

The specification is satisfied, but the customer is not. This happens when requirements don't align with actual needs.

Change Control Failure

Without proper change control processes, every change request seems urgent and gets implemented without evaluation.

REAL-WORLD CHALLENGES

Bad Experiences: Technical Debt

Technical Debt Accumulation

Technical debt refers to the implied cost of rework caused by choosing an easy (limited) solution now instead of using a better approach that would take longer.

Poor Documentation

System design is not well documented and the original designer has left the team.

Code Quality Issues

Source code is not well formatted, which introduces extra maintenance cost.

Late Bug Discovery

Late found bugs result in budget overrun and schedule delays.

Customer Complaints

Customers complain of leaked bugs that weren't caught during testing.

Increasing Complexity

System becomes more and more complex while user satisfaction is getting down.

Budget and Schedule

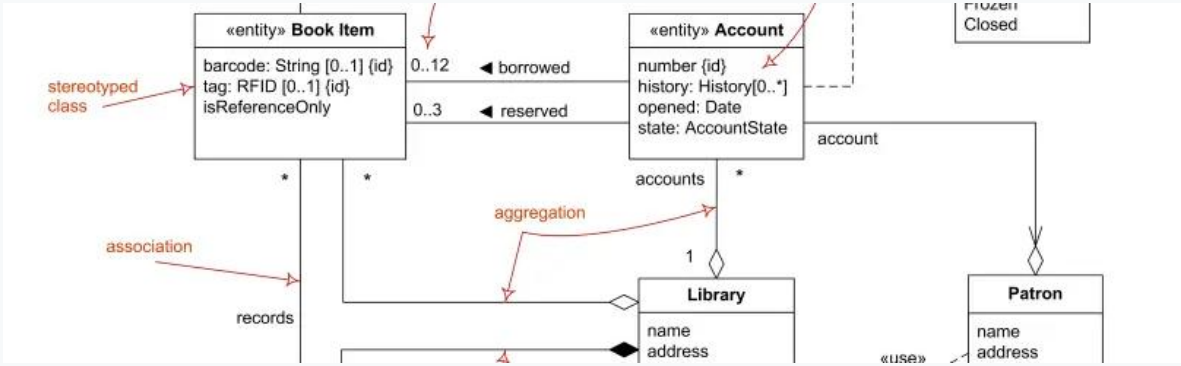
Schedule slipping, budget burnt out — the classic symptoms of project failure.

THE PATH AHEAD

The Way Forward

Systematic Solutions

Need **systematic and disciplined ways** to solve such problems, to make the project a success!



Making Software Engineering a Profession

That's why people study methodologies, create tools to make SE a profession:

- **Methodologies:** Structured approaches
- **Tools:** Automated support
- **Education:** Professional training

Processes
Defined workflows and methodologies

People
Skilled professionals and teamwork

Tools
Automated support technologies

what is SE ?

Definition of “SE” in IEEE

Software Engineering :

1. The application of a systematic , disciplined , quantifiable approach to the development , operation and maintenance of software ; that is , the application of engineering to software.

2. The study of approaches as in 1.

🔑Essence of SE

Max. Gain with Min. Cost!

Optimize value while minimizing resources

A&Q

Frequently Asked Questions about SE

What is software

Computer *programs, data and associated documentation*. Software products may be developed for a particular customer or may be developed for a general market.

What are the attributes of good software?

Good software should deliver the required functionality and performance to the user and should be *maintainable*, *dependable*, *efficient*, and *acceptable*.

What is software engineering?

Software engineering is an *engineering discipline* that is concerned with *all aspects of software production*.

What is the difference between software engineering and computer science?

Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.

A&Q

Frequently Asked Questions about SE

What is the difference between software engineering and system engineering?

System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering. Software engineering is part of this more general process.

What are the key challenges facing software engineering?

Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.

What are the costs of software engineering?

Roughly 60% of software costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.

What are the best software engineering techniques and methods?

While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, *games* should always be developed using *a series of prototypes* whereas *safety critical control systems* require a *complete and analyzable specification* to be developed. You can't, therefore, say that one method is better than another.

Software Myths

Management Myths

Myth: If we get behind schedule, we can add more programmers and catch up.

Reality: Software development is not a mechanistic process like manufacturing. In the words of Brooks “Adding people to a late, software project makes it later” People can be added only in a planned and well-coordinated manner.

Customer Myths

Myth: Project requirements continually change, but change can be easily accommodated because software is flexible.

Reality: The impact of change varies with the time at which it is introduced. As time passes, cost impact grows rapidly. Change can cause upheaval that requires additional resources and major design modification.

Practitioner Myths

Myth: Until I get the program running, I have no way of assessing its quality.

Reality: The formal technical review is one of most effective software quality assurance mechanisms. Software reviews are a “quality filter” that have been found to be more effective than testing for finding certain classes of software errors.

Exercise

1. Which question no longer concerns the modern software engineering (a)
    a. Why does computer hardware cost so much?
    b. Why does software take a long time to finish?
    c. Why does it cost so much to develop a piece of software?
    d. Why can't software errors be removed from products prior to delivery?

2. Today the increased power of the personal computer has brought about an abandonment of the practice of team development of software. (b)
    a. True
    b. False

3. Software is a product and can be manufactured using the same technologies used for other engineering artifacts. (b)
    a. True
    b. False

4. Software deteriorates rather than wears out because (c)
    a. Software suffers from exposure to hostile environments.
    1. Defects are more likely to arise after software has been used often.
    a. Multiple change requests introduce errors in component interactions.
    b. Software spare parts become harder to order.

Exercise

5. Most software continues to be custom built because (d)
   a. Component reuse is common in the software world.
   b. Reusable components are too expensive to use.
   c. Software is easier to build without using someone else's components.
   d. Off-the-shelf software components are unavailable in many application domains.

6. The nature of software applications can be characterized by their information (d)
   a. complexity
   b. content
   c. determinacy
   d. both b and c

7. Change cannot be easily accommodated in most software systems, unless the system was designed with change in mind. (a)
   a. True
   b. False

8. The functionality of most computer systems does not need to be enhanced the lifetime of the system. (b)
   - True
   - False

CHAPTER 1 COMPLETE

Introduction to Software Engineering

Discipline Established

From 1968 NATO conference to SWEBOK

Engineering Principles

Systematic approaches vs. ad-hoc coding

Quality Framework

Product, process, and business perspectives

Software Categories

From system to AI software

Evolution & Crisis

Historical development and challenges

SE Solution

Max gain with min cost

Ready for the next chapter?